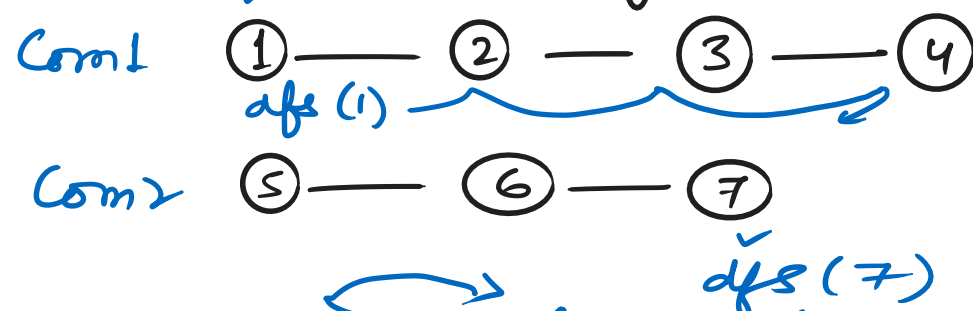


⇒ Disjoint Set data structure (Constant)

Let's consider a graph with 2 components:



BFS traversal will give same time.

Time Complexity?

$O[N+E]$
Linear

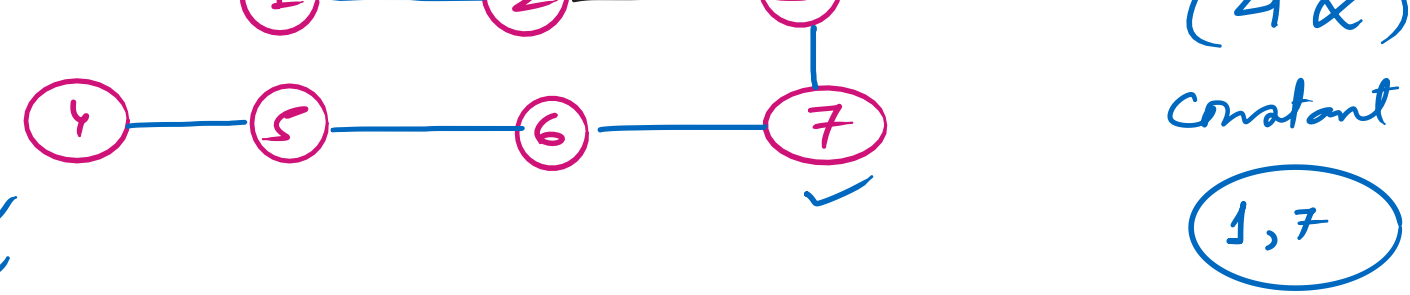
Question:
Do 1 & 7 belong to the same component?
[False/No]

Disjoint Set Use Case?

Check whether two nodes lie in the same component in constant time specifically for graphs changing dynamically.

Example: →

- (1,2) ✓
- (2,3) ✓
- (4,5) ✓
- (6,7) ✓
- (5,6) ✓
- (3,7) ✓



(4x)
constant
1,7

Criteria: At any point of time we should be able to answer whether two nodes belong to the same component or not in Constant Time. (Disjoint Sets)

Disjoint Set : Union By Rank : $\text{union}(u,v)$

(u,v)

(1,2) ✓

(2,3)

(4,5)

(6,7)

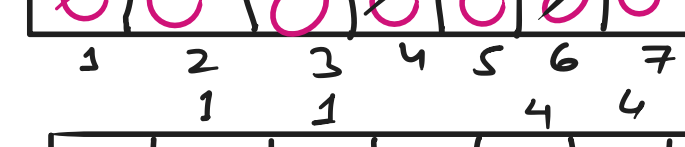
(5,6)

(3,7)

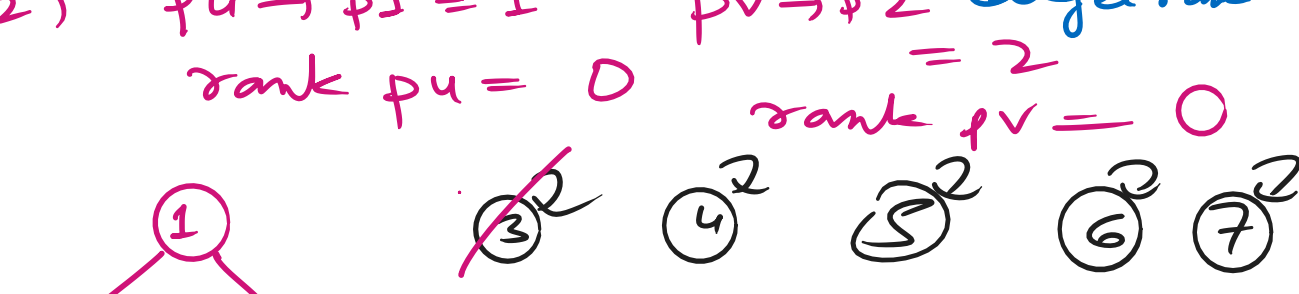
rank



parent



Initially Nodes are connected to itself & itself as parent.



(1,2) $p_u \rightarrow p_1 = 1$ $p_v \rightarrow p_2 = 2$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



(2,3) $p_u = 2 = 1$ $p_v = 3$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



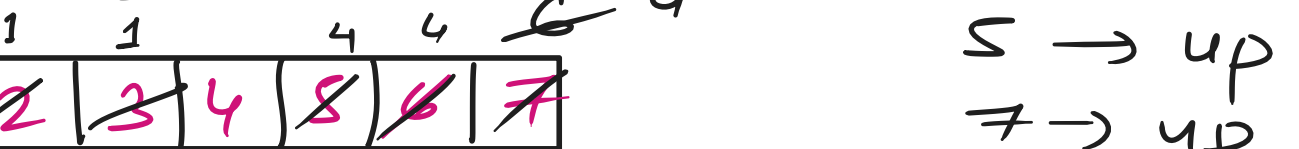
(4,5) $p_u = 4$ $p_v = 5$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



(6,7) $p_u = 6$ $p_v = 7$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



(5,6) $p_u = 4$ $p_v = 6$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



(3,7) $p_u = 3$ $p_v = 7$
 $\text{rank } p_u = 0$ $\text{rank } p_v = 0$



3 & 7 belong to the same? (NO)

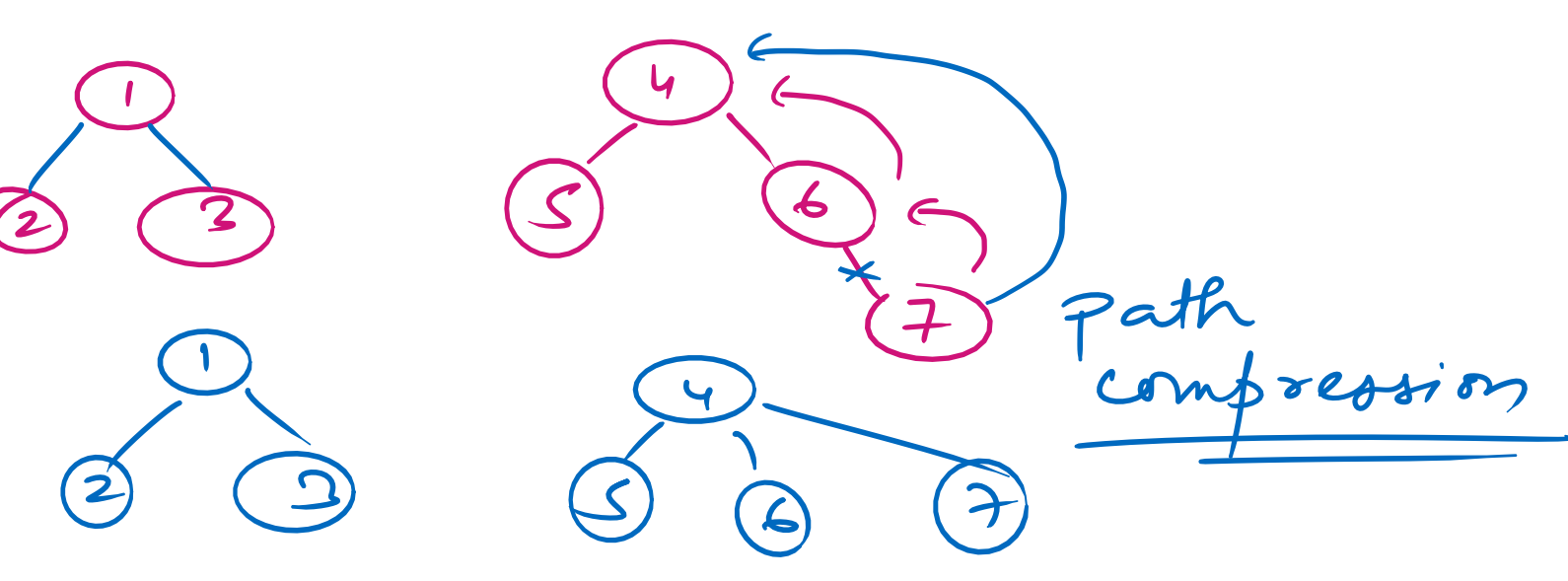
rank



parent

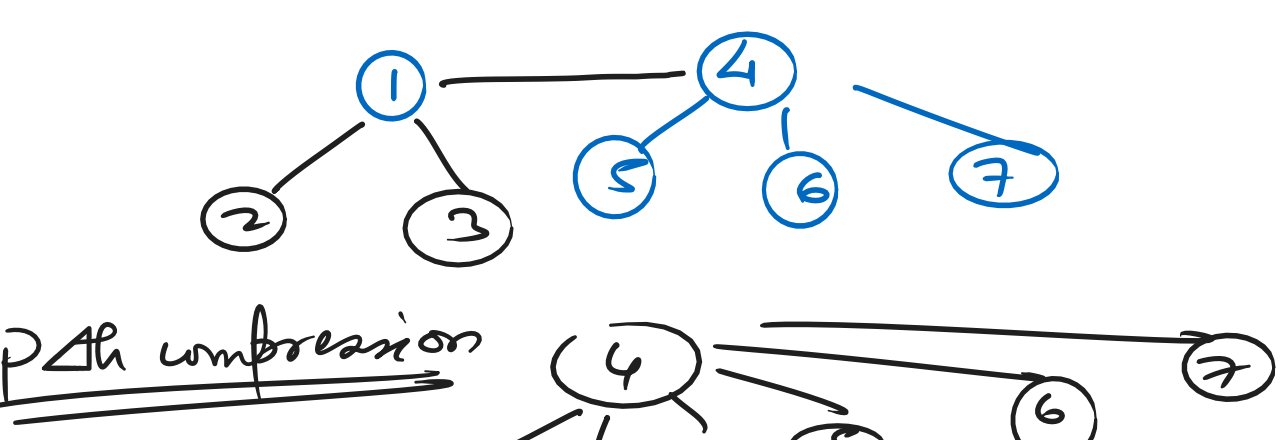


Path Compression: → $\lceil \log N \rceil$



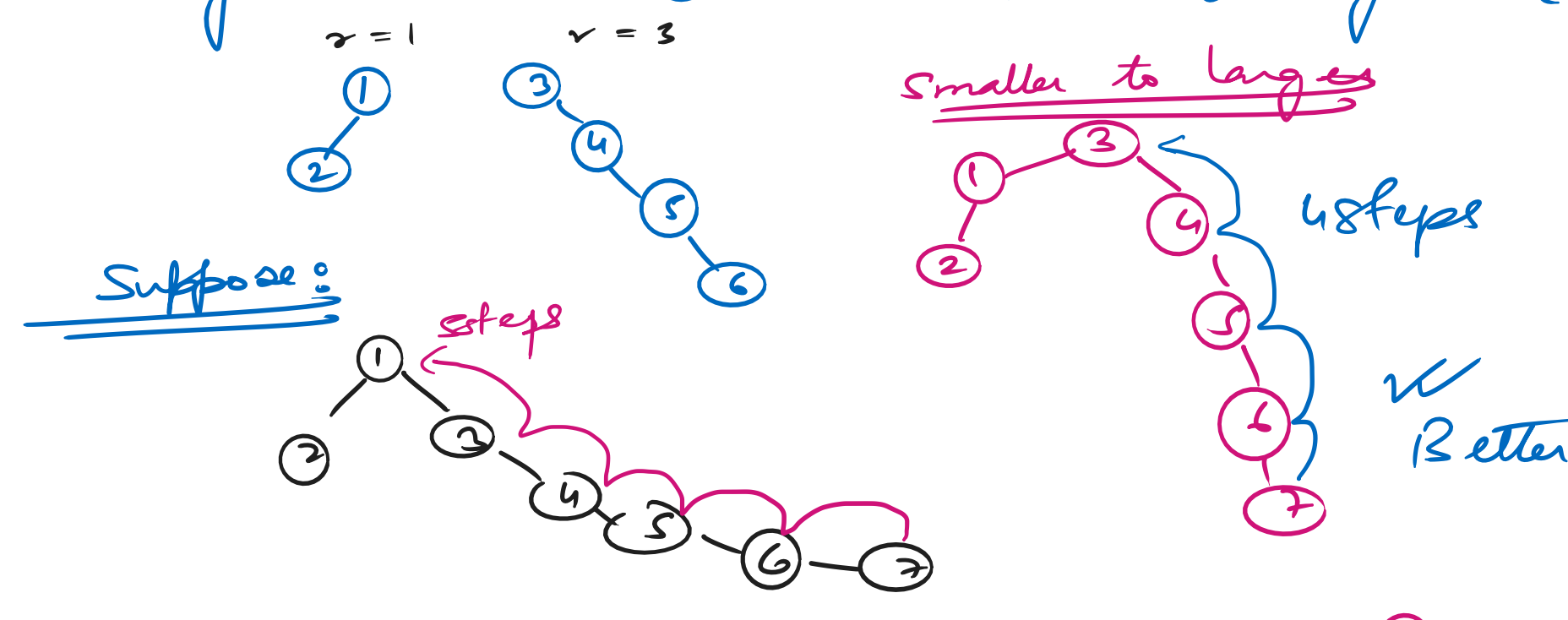
(3,7)

$3 \rightarrow p_u \rightarrow 1$ $\text{rank } p_u = 1$ $7 \rightarrow p_v \rightarrow 4$ $\text{rank } p_v = 2$



Constant → (4x) (u,v) (3,7)
 $O(4x) \rightarrow \text{Inverse Ackermann}$

Why do we connect smaller rank to larger rank?



Disjoint Set → Union by Size: →

(1,2) ✓

(2,3) ✓

(4,5) ✓

(6,7) ✓

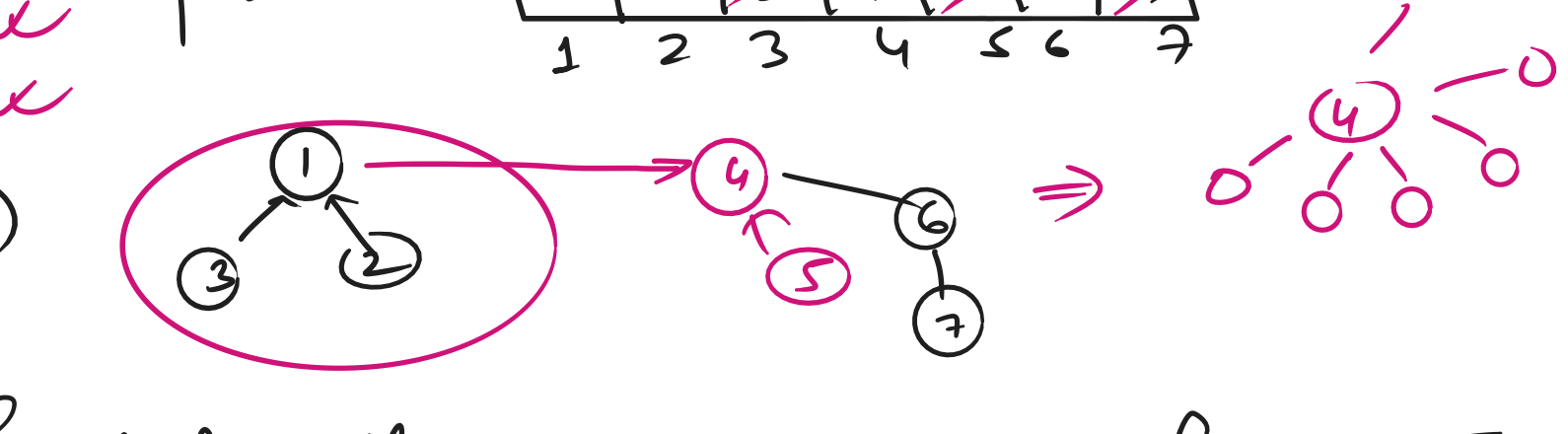
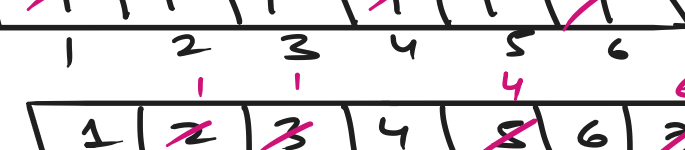
(5,6) ✓

(3,7) ✓

Size



parent

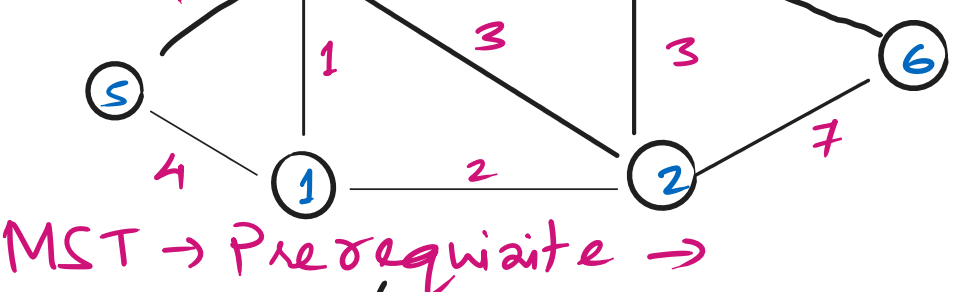


Kruskal's Algorithm : Minimum Spanning Tree

[Joseph Kruskal] → Mathematician

Disjoint Set Data Structure

* Step 1: Sort the edges in ascending order: →



MST → Prerequisite →

N Nodes / N-1 Edges

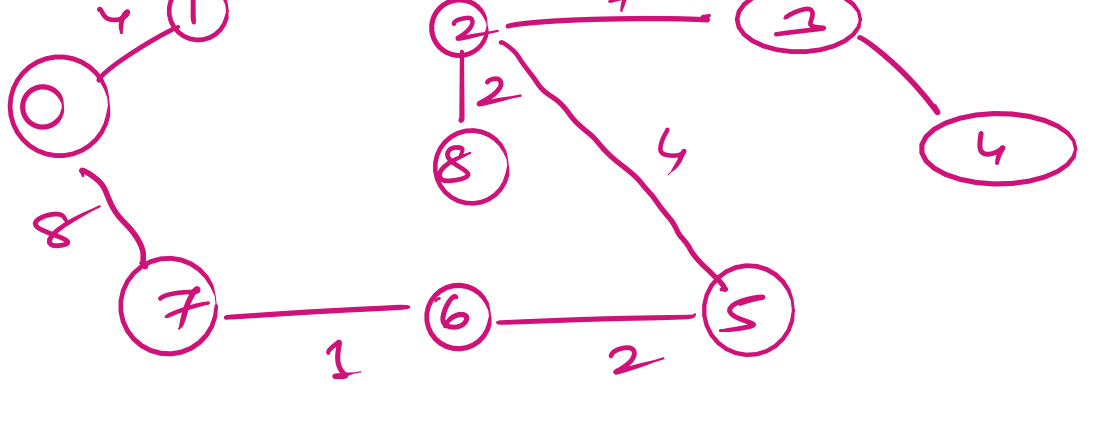
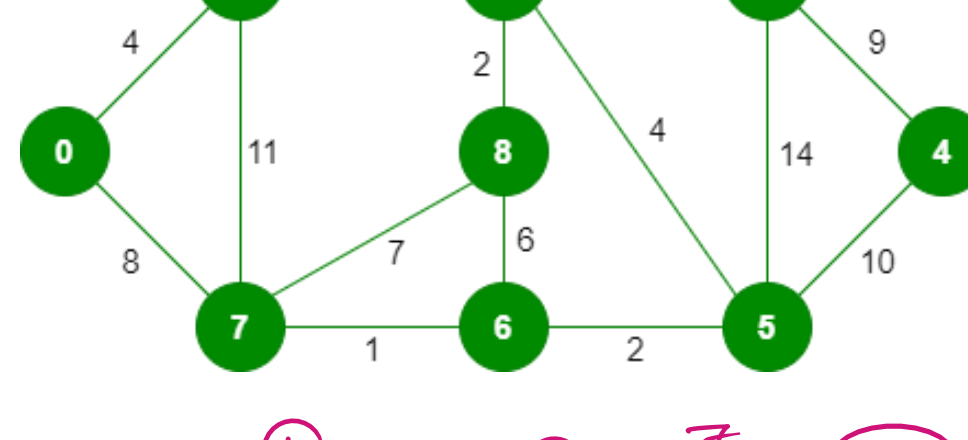
* Find Ultimate Parent * Union By Rank

* If they don't belong to the same component, it will be part of MST.



- $s_1 \rightarrow 1, 1, 4$
- $s_2 \rightarrow 2, 1, 2$
- $s_3 \rightarrow 3, 2, 3$
- $s_4 \rightarrow 4, 1, 5$
- $s_5 \rightarrow 5, 4, 3$
- $s_6 \rightarrow 7, 2, 6$
- $s_7 \rightarrow 8, 3, 6$
- $s_8 \rightarrow 9, 4, 5$

MST → $1+2+3+4+7 = 17$



- wt: 1, 2, 2, 4, 4, 6, 7, 7, 8, 9, 10, 11
- u: 6, 2, 5, 0, 2, 6, 2, 7, 0, 1, 3, 4, 1, 3
- v: 7, 8, 6, 1, 5, 8, 3, 8, 7, 2, 4, 5, 7, 5